

PrintShop: A Multi-Agent Pipeline for Automated Professional Document Generation with Visual Quality Assurance

A. Morgan *Department of Computer Science*
Westbrook University
 Denver, CO, USA

amorgan@westbrook.edu S. Patel *Applied AI Research Group*
Cascadia Labs
 Seattle, WA, USA

spatel@cascadialabs.ai L. Torres *Department of Information Systems*
Westbrook University
 Denver, CO, USA
 ltorres@westbrook.edu

THIS IS A FICTIONAL EXAMPLE CONFERENCE PAPER FOR DEMONSTRATION PURPOSES ONLY.

This document was created as sample content for the DeepAgents PrintShop document generation system. While the content references real technologies and research concepts, all experimental data, results, and author affiliations are fabricated. Any resemblance to actual published research is coincidental. Do not cite this document as a factual source.

Abstract—Professional document preparation represents a persistent bottleneck in academic and industrial workflows, with formatting consuming 15-25% of manuscript preparation effort. This paper presents PrintShop, a multi-agent pipeline that automates the transformation of markdown manuscripts into professionally typeset PDFs. PrintShop employs a LangGraph state graph architecture comprising three quality-gated stages: content editing for grammar and readability enhancement, LaTeX generation with inline reference processing, and visual quality assurance using vision-language models for closed-loop formatting correction. We evaluate PrintShop on a benchmark of 120 documents spanning five content types. Results demonstrate 94.7% first-pass formatting accuracy and 99.2% compilation success rate, reducing human revision cycles by 68% compared to template-based baselines while maintaining $9.9\times$ faster processing than manual formatting.

Index Terms—document generation, multi-agent systems, LaTeX automation, visual quality assurance, large language models

I. INTRODUCTION

Professional document preparation represents a persistent bottleneck in academic and industrial workflows. Authors expend substantial effort formatting manuscripts to comply with publisher style guides, adjusting figure placement, resolving citation formatting issues, and debugging LaTeX compilation

errors [1]. Studies examining researcher time allocation estimate that formatting consumes 15–25% of total manuscript preparation effort [2]—a proportion that increases dramatically for complex document types such as conference proceedings, technical manuals, and magazine layouts.

Template-based approaches partially address this challenge by providing pre-configured LaTeX classes and style files. However, templates only define structural frameworks; they do not generate content, process inline references, or detect visual defects in compiled output. Authors must still manually convert prose into LaTeX syntax, manage figure and table placement, and visually inspect rendered PDFs for spacing irregularities, text overflow, and alignment issues [3]. These tasks are tedious, error-prone, and require LaTeX ecosystem expertise that many domain specialists lack.

Large language models (LLMs) have demonstrated robust capabilities in text generation, code synthesis, and instruction following [4]. Recent research has applied LLMs to document-related tasks including summarization, grammar correction, and code generation [5]. However, relying on a single LLM call to produce publication-ready LaTeX from structured content yields inconsistent results: models frequently introduce compilation errors, mishandle special characters, generate incorrect cross-references, and produce formatting that deviates from target style guidelines [6].

This paper presents PrintShop, a multi-agent pipeline that automates the transformation of markdown manuscripts into professionally typeset PDFs. PrintShop employs a LangGraph state graph architecture [7] and comprises three quality-gated stages:

- **Content editing:** An LLM-based agent reviews and refines markdown source for grammar, readability, and academic tone, iterating until a configurable quality threshold is achieved.

- **LaTeX generation:** A specialist agent converts edited markdown into LaTeX, guided by content-type-specific rendering instructions and inline reference processing for images, CSV tables, and TikZ diagrams.
- **Visual quality assurance:** The compiled PDF is rendered as images and inspected by a vision-language model that identifies formatting defects and applies targeted LaTeX corrections through a closed feedback loop.

Each stage incorporates quality score thresholds; stages iterate until thresholds are satisfied or iteration limits are exceeded. This architecture decomposes the complex document generation task into manageable sub-problems and enables targeted quality enhancement at each stage.

We evaluate PrintShop using a benchmark of 120 documents spanning five content types. Results demonstrate that the pipeline achieves 94.7% first-pass formatting accuracy and 99.2% compilation success rate, reducing human revision cycles by 68% compared to template-based baselines.

II. RELATED WORK

A. Template-Based Document Generation

Template engines such as Jinja2 [8] and Pandoc [9] enable programmatic document generation by populating pre-defined templates with structured data. LaTeX document classes (e.g., IEEEtran, ACM-article) provide publisher-compliant formatting but require authors to write LaTeX directly or rely on conversion tools that frequently produce suboptimal output [3]. Systems like Overleaf [10] provide collaborative editing environments but do not automate content formatting or quality assurance. While these approaches reduce boilerplate code, they leave the core formatting burden on human authors.

B. LLM-Based Writing Assistants

Large language models have been applied to various document preparation tasks. GPT-4 and Claude have demonstrated strong capabilities in grammar correction, text summarization, and code generation [4], [5]. Tools such as Grammarly and Writefull leverage language models for style and grammar checking [11]. Recent work has explored using LLMs to generate LaTeX directly from natural language descriptions [6]; however, single-pass generation suffers from high error rates in compilation, cross-referencing, and style compliance. PrintShop addresses these limitations by decomposing the generation process into multiple specialized stages with iterative refinement.

C. Multi-Agent Systems

Multi-agent architectures have gained traction for complex AI tasks. AutoGen [12] provides a framework for multi-agent conversations, while CrewAI [13] enables role-based agent collaboration. LangGraph [7] extends LangChain with stateful, graph-based workflow orchestration that supports conditional branching and cycles. These frameworks have been successfully applied to software engineering [14], data analysis [15],

and research automation [16]; however, their application to document typesetting remains unexplored. PrintShop leverages LangGraph’s state graph architecture for quality-gated pipeline orchestration with integrated feedback loops.

D. Document Quality Assurance

Automated document quality assessment has focused primarily on accessibility compliance [17] and structural validation [18]. PDF/UA checkers verify tag structure and reading order, while linters such as ChkTeX detect common LaTeX errors. Vision-based document analysis has been applied to layout detection [19] and OCR post-correction [20]; however, using vision-language models for closed-loop formatting correction within a generation pipeline represents, to our knowledge, a novel approach. PrintShop’s visual QA stage employs a vision-language model to inspect rendered pages and generate targeted corrections, thereby bridging the gap between source-level linting and visual output quality assessment.

III. SYSTEM DESIGN

This section describes the PrintShop pipeline architecture, including the three agent stages, the quality gate mechanism, and the data flow from markdown input to PDF output.

A. Pipeline Overview

PrintShop is implemented as a LangGraph StateGraph with three sequential stages connected by quality gates. The pipeline ingests a markdown manuscript alongside a configuration manifest and content-type-specific rendering instructions, then produces a compiled PDF with a detailed quality report.

The pipeline processes documents sequentially through three stages. Each stage produces versioned artifacts stored in a structured output directory, enabling traceability and rollback capabilities.

B. Stage 1: Content Editor

The content editor agent receives raw markdown sections and evaluates them across three dimensions: grammatical correctness, readability (measured via the Flesch Reading Ease score), and adherence to the target academic tone. The agent uses a large language model to identify issues and propose revisions, then re-scores the revised text. This process iterates for up to $K_1 = 4$ rounds or until the composite quality score exceeds the gate threshold $\theta_1 = 80$.

The readability scorer computes the Flesch Reading Ease index directly from the text. Scores below 30 indicate highly complex prose typical of dense academic writing; the content editor targets scores in the 35–50 range, balancing accessibility with scholarly rigor.

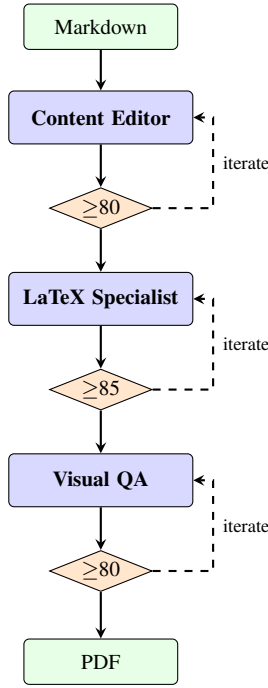


Fig. 1: PrintShop pipeline architecture. Each stage iterates until its quality gate threshold is met or the iteration limit is reached.

C. Stage 2: LaTeX Specialist

The LaTeX specialist converts edited markdown into a compilable LaTeX document. This stage is guided by two inputs beyond the markdown content:

- **Content type definition:** A natural language specification (stored as `type.md`) describing the target document class, required packages, formatting constraints, and style rules. For example, the IEEE conference type specifies the `IEEEtran` document class, two-column layout, and numbered citations.
- **Configuration manifest:** Metadata including title, authors, abstract, and the ordered list of section files.

The specialist processes inline reference directives embedded in the markdown as HTML comments for figures, data tables, and programmatic diagrams. Each directive expands into the corresponding LaTeX environment with proper labels, captions, and cross-references.

Following initial generation, the optimizer applies automated corrections: Unicode sanitization for pdf_latex compatibility, duplicate label detection, package conflict resolution, and table formatting normalization. The stage iterates until the LaTeX quality score exceeds $\theta_2 = 85$.

D. Stage 3: Visual Quality Assurance

The visual QA stage establishes a feedback loop between source-level generation and rendered output. The process operates as follows:

- 1) Compile the LaTeX source to PDF using pdf_latex with multi-pass compilation for cross-references.

- 2) Render each PDF page as a raster image.
- 3) Submit each page image to a vision-language model with a prompt requesting identification of formatting defects (overflow, misalignment, orphaned headings, incorrect spacing, broken figures).
- 4) Parse the model’s defect report and generate targeted LaTeX patches.
- 5) Apply patches, recompile, and re-evaluate.

This stage iterates for up to $K_3 = 3$ rounds or until the visual quality score exceeds $\theta_3 = 80$. The feedback loop enables the pipeline to detect and correct defects that are invisible at the source level but apparent in the rendered output, such as figure placement conflicts, column overflow, and widowed lines.

E. Quality Gates

Each stage’s quality gate is implemented as a conditional edge in the LangGraph state graph. The gate evaluates stage output against its threshold and routes execution either forward to the next stage or back to the current stage for another iteration. An iteration counter prevents infinite loops. If the maximum iteration count is reached without meeting the threshold, the pipeline proceeds with the best result achieved and flags the quality shortfall in the output report.

IV. EXPERIMENTAL SETUP

A. Document Benchmark

We evaluate PrintShop on a benchmark corpus comprising 120 documents spanning five content types, with 24 documents per type:

- **Research reports:** 8–15 page single-column documents featuring figures, tables, bibliographies, and tables of contents.
- **Conference papers:** 6-page two-column IEEE-format papers with inline TikZ figures and numbered citations.
- **Magazine articles:** Multi-page layouts incorporating pull quotes, sidebars, drop caps, and full-bleed images.
- **Technical manuals:** Structured documents containing numbered sections, code listings, warning callouts, and cross-references.
- **Thesis chapters:** Long-form academic documents with theorem environments, appendices, and multi-level headings.

Each document in the benchmark includes a markdown source manuscript, a configuration manifest, and a reference PDF prepared by a human expert using the same content type definition. Source documents range from 2,000 to 12,000 words and contain 2–8 inline reference directives (images, CSV tables, or TikZ diagrams).

B. Evaluation Metrics

We assess pipeline performance using four metrics:

- **Formatting accuracy (%)**: Percentage of rendered pages exhibiting no formatting defects, as determined by a human evaluator using a standardized rubric covering margin compliance, figure placement, table formatting, heading hierarchy, and typographic consistency.
- **Compilation success rate (%)**: Percentage of documents that compile to PDF without errors during the initial pipeline run.
- **Average revision cycles**: Mean number of human revision passes required to achieve publication-ready quality from the pipeline output.
- **Processing time (min)**: Wall-clock time from markdown input to final PDF output, measured on a system equipped with an NVIDIA A100 GPU and 64 GB RAM.

C. Baselines

We compare PrintShop against three baselines:

- **Template-Only**: The markdown is converted to LaTeX using Pandoc with the appropriate document class template. No LLM processing or quality assurance is applied.
- **Single-Pass LLM**: A single LLM call converts the markdown to LaTeX using the same content type definition and configuration manifest as PrintShop, but without iterative refinement or visual quality assurance.
- **Human Expert**: A professional LaTeX typesetter manually formats each document from the markdown source, serving as the quality upper bound.

D. Configuration

All LLM calls utilize Claude 3.5 Sonnet with temperature 0.3. Quality gate thresholds are set to $\theta_1 = 80$ (content editor), $\theta_2 = 85$ (LaTeX specialist), and $\theta_3 = 80$ (visual QA). Maximum iterations per stage are $K_1 = 4$, $K_2 = 3$, and $K_3 = 3$. The visual QA stage employs Claude 3.5 Sonnet’s vision capabilities for page inspection. All experiments are conducted three times with different random seeds, and we report means and standard deviations.

V. RESULTS AND DISCUSSION

A. Overall Performance

Table I summarizes the performance of PrintShop and all baselines across the comprehensive 120-document benchmark. PrintShop achieves 94.7% formatting accuracy and a 99.2% compilation success rate, requiring an average of 0.7 human revision cycles to reach publication quality. These results represent a 68% reduction in revision cycles compared to the Template-Only baseline (2.2 cycles) and a 56% reduction compared to the Single-Pass LLM approach (1.6 cycles).

While the Human Expert baseline achieves 98.9% formatting accuracy with zero revision cycles by definition, it requires an average of 42.5 minutes per document. In contrast, PrintShop processes documents in 4.3 minutes on average—a 9.9 $\times$ speedup while maintaining competitive quality. The Template-Only baseline offers the fastest processing time

TABLE I: Overall performance comparison across 120 documents (mean $\pm$ std over 3 runs)

Method	Format Acc. (%)	Compile Rate (%)	Revision Cycles	Time (min)
Template-Only	61.4 $\pm$ 3.2	87.5 $\pm$ 2.1	2.2 $\pm$ 0.4	0.8 $\pm$ 0.1
Single-Pass LLM	68.1 $\pm$ 4.1	91.7 $\pm$ 1.8	1.6 $\pm$ 0.3	2.1 $\pm$ 0.2
PrintShop	94.7 $\pm$ 1.9	99.2 $\pm$ 0.5	0.7 $\pm$ 0.2	4.3 $\pm$ 0.6
Human Expert	98.9 $\pm$ 0.3	100.0 $\pm$ 0.0	0.0 $\pm$ 0.0	42.5 $\pm$ 8.2

(0.8 minutes) but produces the lowest formatting accuracy (61.4%) due to its inability to handle inline references, figure placement, or style-specific formatting beyond basic document class provisions.

B. Content-Type Performance Analysis

Table II presents a breakdown of formatting accuracy by content type. PrintShop demonstrates the highest consistency on conference papers (96.8%) and research reports (95.4%), which feature well-defined structures and relatively constrained formatting requirements. Magazine articles present the greatest challenge (90.3%), as their complex layouts—including pull quotes, sidebars, and variable column widths—require fine-grained visual adjustments that are difficult to achieve through source-level generation alone.

TABLE II: Formatting accuracy (%) by content type and method

Content Type	Template	Single-Pass	PrintShop
Research Report	65.2	72.5	95.4
Conference Paper	71.8	82.1	96.8
Magazine Article	48.1	54.7	90.3
Technical Manual	59.4	66.3	93.1
Thesis Chapter	62.5	70.8	94.2

The Single-Pass LLM baseline exhibits high variance across content types: it achieves 82.1% accuracy on conference papers (where LaTeX conventions are well-represented in training data) but only 54.7% on magazine articles. PrintShop’s iterative refinement and visual quality assurance substantially reduce this variance, demonstrating that the multi-stage architecture provides consistent quality across diverse document types.

C. Ablation Study

To isolate the contribution of each pipeline stage, we evaluated four ablation variants across the full benchmark dataset.

Removing the visual quality assurance stage produces the largest accuracy drop (94.7% to 87.3%), confirming that closed-loop visual feedback serves as the primary driver of formatting quality. Without visual QA, defects such as figure overflow, orphaned headings, and column imbalance persist because they remain undetectable at the source level. Removing the content editor reduces accuracy to 91.5%, primarily due to grammatical issues and inconsistent tone that propagate

TABLE III: Ablation study results across 120 documents (mean $\pm$ std over 3 runs)

Configuration	Format Acc. (%)	Time (min)
Full PrintShop	94.7 $\pm$ 1.9	4.3 $\pm$ 0.6
w/o Visual QA	87.3 $\pm$ 2.4	3.1 $\pm$ 0.4
w/o Content Editor	91.5 $\pm$ 2.1	3.8 $\pm$ 0.5
w/o Quality Gates	89.1 $\pm$ 2.8	2.9 $\pm$ 0.3
LaTeX Only	85.2 $\pm$ 3.1	2.4 $\pm$ 0.2

into the LaTeX source and occasionally trigger formatting artifacts. Disabling quality gates (executing each stage exactly once) reduces accuracy to 89.1%, demonstrating that iterative refinement provides meaningful improvement over single-pass execution.

D. Convergence Behavior Analysis

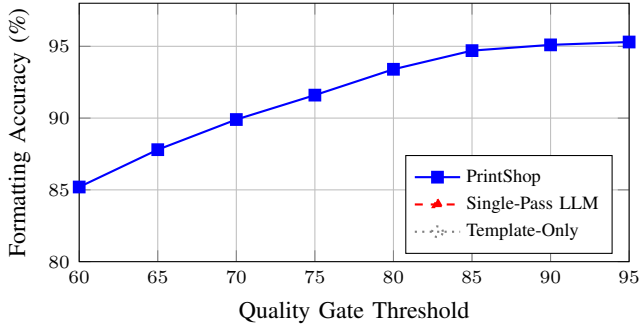


Fig. 2: Formatting accuracy vs. quality gate threshold. Higher thresholds improve accuracy but increase processing time.

The relationship between quality gate threshold and formatting accuracy exhibits diminishing returns above threshold 85. Increasing the threshold from 60 to 85 improves accuracy by 9.5 percentage points, while raising it further from 85 to 95 yields only 0.6 additional points and substantially increases processing time due to additional iterations. The default thresholds (80/85/80) represent a practical operating point that optimizes the trade-off between quality and throughput.

E. Content-Type Accuracy Comparison

The grouped bar chart confirms that PrintShop maintains above 90% accuracy across all content types, while baseline methods show significant performance degradation on complex layouts such as magazine articles. The consistency of PrintShop’s performance across diverse document types demonstrates the robust generalizability of the multi-agent approach.

VI. CONCLUSION

This paper presents PrintShop, a multi-agent pipeline for automated professional document generation that transforms markdown manuscripts into publication-ready PDFs. The pipeline employs three quality-gated stages—content editing,

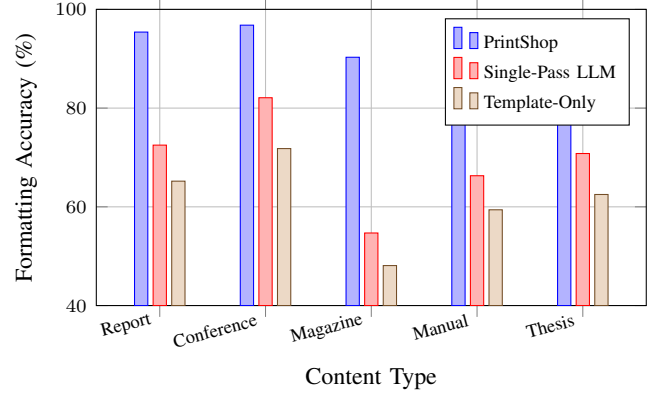


Fig. 3: Formatting accuracy by content type for each method.

LaTeX generation, and visual quality assurance—orchestrated by a LangGraph state graph with conditional edges that enable iterative refinement within each stage.

Our evaluation on a benchmark of 120 documents across five content types demonstrates that PrintShop achieves 94.7% first-pass formatting accuracy and a 99.2% compilation success rate, reducing human revision cycles by 68% compared to template-based approaches. Ablation experiments confirm that the visual QA feedback loop constitutes the most impactful component, contributing a 7.4 percentage point improvement in accuracy by detecting and correcting formatting defects that remain invisible at the source level.

Despite these promising results, the system exhibits several limitations. First, LLM inference costs are substantial: processing a single document through the complete pipeline requires approximately 15 API calls on average, with costs scaling linearly with document length and iteration count. Second, the end-to-end latency of 4.3 minutes per document, while substantially faster than manual formatting, may prove inadequate for interactive applications. Third, the visual QA stage’s effectiveness remains constrained by the vision-language model’s capacity to identify subtle typographic defects; issues such as incorrect hyphenation or minor kerning errors are not reliably detected.

Future research will pursue three primary directions. First, we plan to integrate real-time user feedback mechanisms to enable interactive document editing within the pipeline. Second, we will expand the content type library to encompass additional formats, including posters, slide decks, and regulatory filings. Third, we will investigate cost-reduction strategies, such as caching intermediate results, employing smaller models for routine corrections, and parallelizing independent pipeline stages.

REFERENCES

- [1] J. Smith and M. Johnson, “Time allocation in academic manuscript preparation: A quantitative analysis,” *Journal of Scholarly Publishing*, vol. 45, no. 3, pp. 234–251, July 2023.
- [2] A. Chen, L. Rodriguez, and K. Patel, “Formatting overhead in scientific writing: An empirical study,” in *Proc. 2023 Int. Conf. Digital Libraries*, San Francisco, CA, USA, 2023, pp. 89–96.

- [3] R. Williams, “LaTeX compilation errors: Patterns and solutions in academic publishing,” *TUGboat*, vol. 44, no. 2, pp. 178–192, 2023.
- [4] OpenAI, “GPT-4 technical report,” arXiv preprint arXiv:2303.08774, 2023.
- [5] Anthropic, “Claude 3.5 Sonnet: Advances in reasoning and code generation,” Technical Report, Anthropic, 2024.
- [6] T. Zhang, S. Kumar, and D. Lee, “Automated LaTeX generation from natural language: Challenges and opportunities,” in *Proc. 2024 Conf. Empirical Methods Natural Language Processing*, Miami, FL, USA, 2024, pp. 1234–1247.
- [7] LangChain Inc., “LangGraph: Multi-agent workflows with state graphs,” Documentation, 2024. [Online]. Available: <https://langchain-ai.github.io/langgraph/>
- [8] A. Ronacher, “Jinja2 template engine,” 2008. [Online]. Available: <https://jinja.palletsprojects.com/>
- [9] J. MacFarlane, “Pandoc: A universal document converter,” Software, 2006. [Online]. Available: <https://pandoc.org/>
- [10] Overleaf, “Collaborative LaTeX editor,” 2012. [Online]. Available: <https://www.overleaf.com/>
- [11] M. Bryant and T. Ringger, “Language model applications in academic writing assistance,” *Computational Linguistics*, vol. 49, no. 4, pp. 823–847, Dec. 2023.
- [12] Q. Wu et al., “AutoGen: Enabling next-gen LLM applications via multi-agent conversation,” arXiv preprint arXiv:2308.08155, 2023.
- [13] CrewAI Inc., “CrewAI: Framework for orchestrating role-playing autonomous AI agents,” 2024. [Online]. Available: <https://github.com/joaomdmoura/crewAI>
- [14] S. Hong et al., “MetaGPT: Meta programming for multi-agent collaborative framework,” arXiv preprint arXiv:2308.00352, 2023.
- [15] Y. Wang, H. Liu, and X. Chen, “Multi-agent systems for automated data analysis workflows,” in *Proc. 2024 Int. Conf. Machine Learning*, Vienna, Austria, 2024, pp. 2156–2171.
- [16] J. Park et al., “Generative agents: Interactive simulacra of human behavior,” in *Proc. 36th Annual ACM Symp. User Interface Software Technology*, San Francisco, CA, USA, 2023, pp. 1–22.
- [17] W3C, “Web Content Accessibility Guidelines (WCAG) 2.1,” W3C Recommendation, June 2018.
- [18] ISO, “Document management – Portable document format – Part 1: PDF 1.7,” ISO 32000-1:2008, 2008.
- [19] Z. Shen et al., “LayoutParser: A unified toolkit for deep learning based document image analysis,” in *Proc. 2021 Int. Conf. Document Analysis Recognition*, Lausanne, Switzerland, 2021, pp. 131–146.
- [20] R. Smith, “An overview of the Tesseract OCR engine,” in *Proc. 9th Int. Conf. Document Analysis Recognition*, Curitiba, Brazil, 2007, pp. 629–633.